

Práctica 1 - GitHub en equipo

Hoy no se programa. Hoy aprendéis a trabajar cinco personas en el mismo repositorio sin pisaros: tareas, ramas, pull requests, revisiones y conflictos. Todo desde el navegador, sin instalar nada. Al terminar, el repositorio **club-mayores** tendrá vuestros nombres, vuestras tareas y el borrador de vuestra primera decisión técnica.

El plan

MISIÓN	QUÉ
0	Entrar en la organización
1	Explorar el repositorio (búsqueda del tesoro)
2	Tu primera issue
3	Tu primera pull request
4	Revisar la de un compañero
5	Merge y conflictos
6	El borrador de tu decisión
+	Retos extra, si te sobra tiempo

CÓMO SE TRABAJA HOY

- No hay reloj: cada uno avanza a su ritmo. Las misiones van en orden: no saltes. Lo que no termines hoy, lo terminas en casa antes del próximo miércoles.
- Al final de cada misión hay un **punto de control**. No sigas hasta tenerlo todo marcado.
- Las marcas rojas **1** de las capturas corresponden a los pasos con el mismo número.

ATASCADO: PREGUNTA A 3 ANTES QUE AL PROFE

1. Esta hoja y `GUIA_GITHUB.md` del repositorio.
2. Internet o una IA (y compruébalo).
3. Un compañero.

Si aun así no sale, entonces al profe.

Antes de empezar: cómo se llaman las cosas

PALABRA	QUÉ ES	LO MISMO EN NUESTRA APP
Repositorio	La carpeta del proyecto con toda su historia	La asociación y todo lo que ha pasado en ella
Issue	Una tarea o un problema, con número y responsable	Un punto del orden del día
Commit	Una foto de unos cambios: quién, cuándo, por qué	Una línea en la libreta del tesorero
Rama	Una línea de trabajo paralela. <code>main</code> es lo que funciona	Un borrador de actividad sin publicar

Pull request	"Propongo estos cambios, revisadlos"	Un socio propone una excursión; el admin la acepta
Merge	Unir los cambios de una rama a <code>main</code>	Publicar la actividad

EL CICLO QUE VAS A HACER HOY DOS VECES

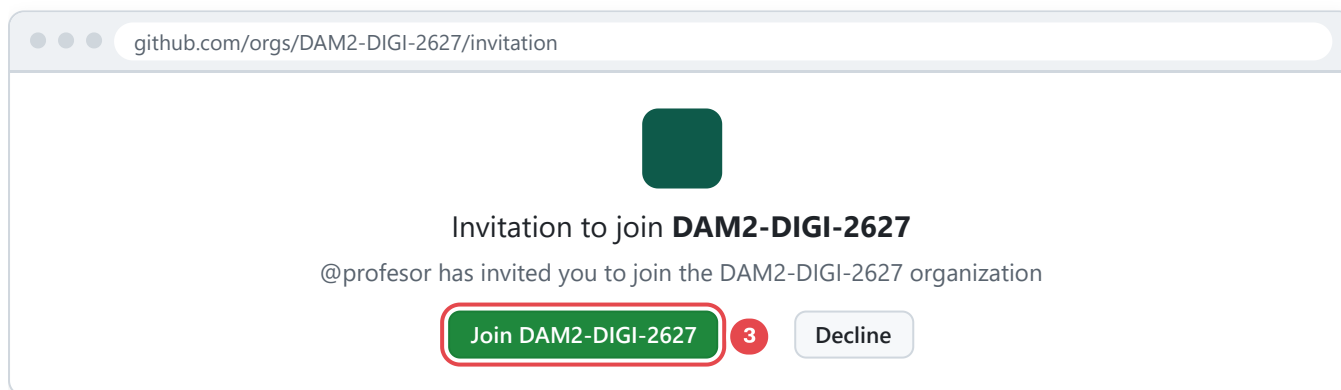
issue → **rama** → **commit** → **pull request** → **revisión** → **merge** → **issue cerrada**

Una advertencia honesta. Muchas cosas de hoy se pueden hacer más rápido por otro camino, y seguramente sabes hacerlo. No se trata de ir rápido, sino de hacerlo como se hace en un equipo, para que el lunes que haya código de verdad no haya sustos. GitHub cambia su interfaz de vez en cuando: si un botón no está exactamente donde dice la captura, busca uno con un nombre parecido.

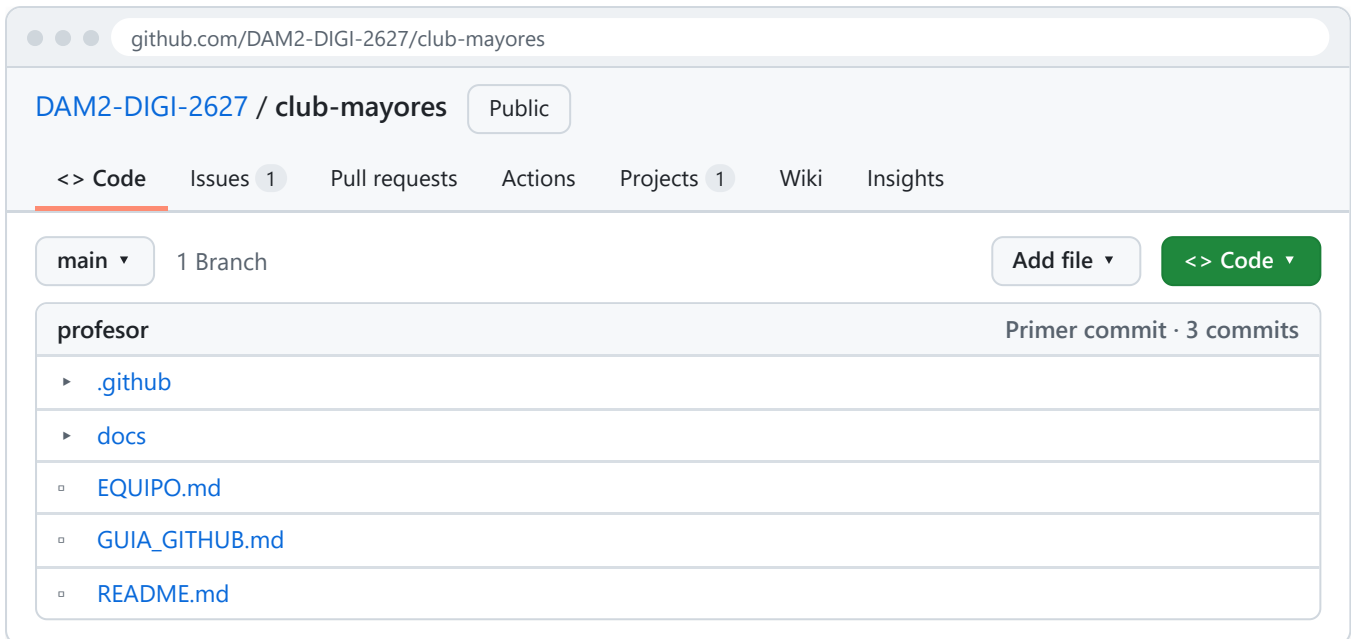
0 Entrar en la organización

Objetivo: ser miembro de la organización del módulo y ver el repositorio club-mayores.

- 1 Inicia sesión en **github.com**. ¿No tienes cuenta? Créala ahora con un correo que vayas a seguir usando. Elige un nombre de usuario serio: lo verán tus futuras empresas.
- 2 Dile tu nombre de usuario al profe. Te llegará una invitación por correo. También la puedes ver en `github.com/orgs/DAM2-DIGI-2627/invitation` (si la organización se llama distinto, el profe te dará el nombre).
- 3 Pulsa `Join DAM2-DIGI-2627` **3**.
- 4 Entra en el repositorio **club-mayores**. Deberías ver algo como la segunda captura.



La invitación a la organización.



El repositorio. Arriba, las pestañas que vas a usar hoy: Code, Issues, Pull requests y Projects.

PUNTO DE CONTROL 0

- ☐ Soy miembro de la organización (mi foto sale en la pestaña People de la organización).
- ☐ Veo el repositorio club-mayores con los ficheros de la captura.

1 Explorar el repositorio

Objetivo: saber qué hay en el repositorio y dónde buscar cuando tengas una duda. Es una búsqueda del tesoro.

Responde a estas preguntas en tu cuaderno. Todas las respuestas están en `README.md`, `GUIA_GITHUB.md`, `EQUIPO.md` o en la carpeta `docs/`. Las soluciones están en la última página: no mires hasta terminar.

1. ¿Qué función **no** va a tener la app, y por qué?
2. ¿Cuándo se considera superado el hito 2?
3. Si subes una contraseña por error y la borras en un commit nuevo, ¿ya está arreglado? ¿Por qué?
4. ¿Qué comando de Git hay que usar "siempre que dudes"?
5. ¿Qué escribes en la descripción de una pull request para que la issue se cierre sola al aceptarla?
6. ¿Quién es el único miembro que aparece ya en `EQUIPO.md`, y qué sabe hacer bien?
7. En la plantilla de decisiones (`docs/decisiones/0000-plantilla.md`), ¿qué fuente **no** vale?
8. ¿Qué hay que hacer en las entrevistas con los nombres de las personas? (Mira en `docs/entrevistas/`.)

Truco: pulsa la tecla **t** dentro del repositorio para buscar ficheros por nombre, y la tecla **/** para buscar texto en todo GitHub.

PUNTO DE CONTROL 1

- ☐ Tengo las 8 respuestas escritas en el cuaderno.

2 Tu primera issue

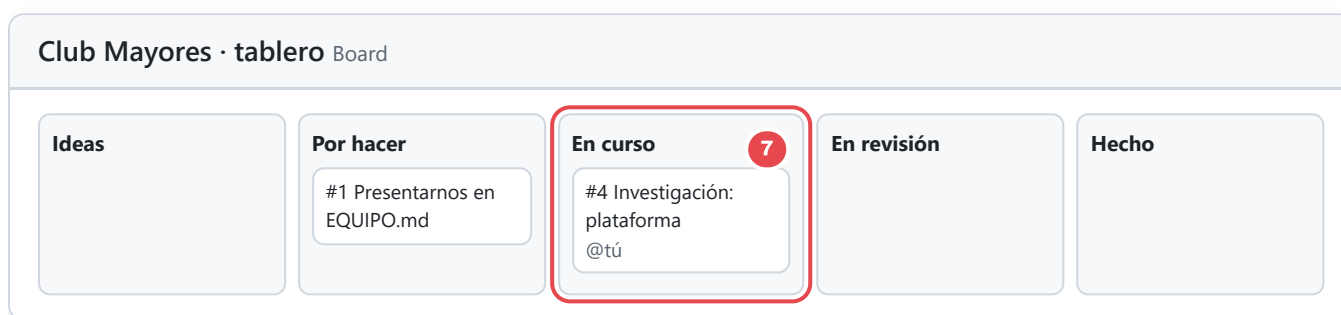
Objetivo: crear la tarea de tu investigación para la semana que viene, asignártela y ponerla en el tablero.

Primero, poneos de acuerdo en 1 minuto: **cada uno se queda una investigación.**

	TEMA	PREGUNTA QUE TIENES QUE RESPONDER	FICHERO DE TU DECISIÓN
A	Plataforma	Web instalable (PWA), Android nativo o Flutter/React Native: ¿con cuál se une uno con un enlace y recibe avisos en Android y en iPhone con menos obstáculos? ¿Cómo programaríamos en clase, donde no se puede instalar nada?	0001-plataforma.md
B	Datos y alojamiento	Servicio gestionado (Supabase, Firebase...) o servidor propio: ¿dónde viven los datos, qué da gratis y qué pasa el día que se supera?	0002-datos.md
C	Avisos	¿Cómo llega una notificación a un Android y a un iPhone desde una web? ¿Qué tiene que hacer el usuario? ¿Qué hacemos con quien no la instale?	0003-avisos.md
D	Accesibilidad	¿Qué dicen las WCAG 2.2 AA en concreto (texto, contraste, botones)? Trae una app bien hecha para mayores y otra mal hecha, con capturas.	0004-accesibilidad.md
E	Datos personales	¿Qué datos necesitamos de verdad? ¿Qué exige el RGPD? ¿Cómo se pide el consentimiento a una persona mayor sin un texto legal ilegible?	0005-datos-personales.md

Si sois cuatro, quien lleve A se encarga también de C.

- 1 Pestaña **Issues** **1** → botón **New issue** **2**.
- 2 Elige la plantilla **Investigación** **3**. Las plantillas rellenan la issue con la estructura que el equipo ha acordado.
- 3 **Título:** completa el que viene, por ejemplo **Investigación: plataforma** **4**.
- 4 En el texto, debajo de "Pregunta que hay que responder", pega tu pregunta de la tabla. En "Fecha límite", pon la fecha de la próxima clase.
- 5 En la columna derecha **5**:
 - **Assignees** → *Assign yourself*.
 - **Labels** → comprueba que tiene **investigación**.
 - **Projects** → elige el tablero del proyecto.
- 6 **Create** (o **Submit new issue**). Apunta el número que te da GitHub, por ejemplo **#4**. Lo necesitarás en la misión 6.
- 7 Pestaña **Projects** → abre el tablero y arrastra tu tarjeta a **En curso** **7**.



El tablero: tu tarjeta, en "En curso".

¿Por qué así? En el tablero, cualquiera (el profe también) sabe en todo momento quién hace qué y cómo va, sin preguntar. Una tarea sin responsable es una tarea que no hará nadie.

PUNTO DE CONTROL 2

- ☐ Mi issue existe, me la he asignado y tiene la etiqueta `investigación`.
- ☐ Está en el tablero, en "En curso". He apuntado su número.

3 Tu primera pull request

Objetivo: añadir tu fila a EQUIPO.md sin tocar main directamente, proponiéndolo en una pull request.

- 1 Pestaña `Code` → abre EQUIPO.md → pulsa el lápiz `Edit this file`.
- 2 Añade tu fila **al final de la tabla**, con el mismo formato que la del profe:


```
| Lucía Pérez | @luciaperez | A · Plataforma | Explicar las cosas con dibujos |
```
- 3 Pulsa `Commit changes...`. Se abre una ventana.
- 4 **Commit message:** Añade a Lucía al equipo (con tu nombre).
- 5 **Experimento:** fijate en la opción *Commit directly to the main branch*. No te deja usarla (o te avisará de que la rama está protegida). Nadie puede tocar `main` sin revisión, ni siquiera tú.
- 6 Marca *Create a new branch for this commit and start a pull request* y llama a la rama `equipo-tunombre`. → `Propose changes`.

- 7 Se abre la página **Open a pull request**. La descripción ya trae la plantilla del equipo. Rellénala:
- **Qué cambia:** "Añado mi fila al equipo".
 - Donde pone `Closes #`, cámbialo por `Parte de #1` 7. No ponemos `Closes` porque la issue #1 no está terminada hasta que entréis los cinco.
 - Marca las casillas de **Comprobado** que sean ciertas.
- 8 Columna derecha → **Reviewers** ⚙ → elige al **compañero sentado a tu derecha** (el último elige al primero) 8.
- 9 **Create pull request** 9. En el tablero, mueve la tarjeta #1 a "En revisión" si nadie lo ha hecho.

club-mayores / EQUIPO.md

```
| Nombre | Usuario | Investigación | Algo que sé  
hacer bien |  
|---|---|---|---|  
| Gabriel Mateos (profesor) | @gabrirobleda | - |  
Hacer preguntas incómodas |  
| Lucía Pérez | @luciaperez | A · Plataforma |  
Explicar con dibujos |
```

2 Cancel changes 3 Commit changes...

Editar el fichero desde la web.

Commit changes

Commit message 4

Añade a Lucía al equipo

Extended description

Add an optional extended description...

☐ Commit directly to the `main` branch 5

Protected by a ruleset

☒ Create a **new branch** for this commit and start a pull request 6

equipo-lucia

Cancel Propose changes

La ventana del commit: main está protegida.

Open a pull request

base: `main` ← compare: `equipo-lucia` ✓ Able to merge.

Añade a Lucía al equipo

Qué cambia

Añado mi fila al equipo.

Parte de #1 7

Comprobado

- [x] Hace una sola cosa
- [x] No sube contraseñas, claves, .env ni datos reales

Create pull request 9

Reviewers 8

@mario

Assignees

No one

Labels

None yet

Projects

None yet

Abrir la pull request: descripción, revisor y botón final.

¿Por qué así? Si cada uno escribe directamente en `main`, en una semana nadie sabe qué ha cambiado ni por qué, y el primer error rompe el trabajo de todos. Con la pull request, cada cambio lo ve al menos otra persona antes de entrar.

PUNTO DE CONTROL 3

- ☐ Mi pull request está abierta, desde la rama `equipo-tunombre` hacia `main`.
- ☐ Pone "Parte de #1" y tiene un revisor asignado.

4 Revisar la de un compañero

Objetivo: hacer tu primera revisión de código de verdad. No se aprueba sin mirar.

- 1 Pestaña `Pull requests`. Busca la que te ha asignado tu compañero de la izquierda. Truco: en el buscador escribe `is:open review-requested:@me`.
- 2 Abre la pestaña `Files changed` 2. Verás en verde las líneas añadidas.
- 3 Revisa con esta lista:
 - ¿Ha tocado solo lo que tenía que tocar? (Una PR = una cosa.)
 - ¿La fila tiene las cuatro columnas y el formato correcto?
 - ¿El título y la descripción de la PR explican el cambio?
- 4 **Deja al menos un comentario en una línea**, aunque sea para decir algo bueno. Pasa el ratón por la línea y pulsa el `+` azul 4. Escribe y pulsa `Start a review`.
- 5 Arriba a la derecha, `Review changes` (en algunas versiones, `Submit review`) 5. Elige:
 - **Approve** si está bien.
 - **Request changes** si hay que corregir algo. Explica qué.→ `Submit review`.
- 6 Si te han pedido cambios a ti: vuelve a editar el fichero **en tu rama** (en la PR, `Files changed` → ... → `Edit file`) y haz commit en la misma rama. La PR se actualiza sola.

Añade a Lucía al equipo #5

Conversation Commits 1 Checks Files changed 1 2

1 file changed

EQUIPO.md

7 | Gabriel Mateos (profesor) | @gabriroblada | - | Hacer...

8  | Lucía Pérez | @luciaperez | A · Plataforma | Explicar... 4

Bien. ¿"Explicar con dibujos" podría ser tu rol en Producto? 😊

Add single comment Start a review

Review changes 5

Finish your review

Leave a comment

☐ Comment

☒ Approve

☐ Request changes

Submit review

Revisar: comentario en una línea y aprobación.

¿Por qué así? En este proyecto, revisar cuenta tanto como programar, y GitHub guarda quién revisó a quién. Una aprobación sin comentarios en una PR grande es sospechosa. Aprender a leer código ajeno es lo que más os va a servir cuando la IA escriba buena parte del código.

PUNTO DE CONTROL 4

- ☐ He dejado al menos un comentario en una línea de la PR de mi compañero.
- ☐ La he aprobado (o he pedido cambios explicando qué).
- ☐ Mi propia PR tiene una revisión de mi compañero.

5 Merge y conflictos

Objetivo: meter tu cambio en main. Y resolver tu primer conflicto, que va a aparecer a propósito.

5.1 · El merge

- 1 Vuelve a tu PR, pestaña **Conversation**, y baja hasta el final.
- 2 **Experimento:** si todavía no te han aprobado, verás *Merging is blocked · Review required* **A**. La regla funciona.
- 3 Cuando tengas la aprobación, pulsa **Merge pull request** → **Confirm merge** **B**. Luego **Delete branch**: la rama ya no hace falta.

**Review required**
At least 1 approving review is required by reviewers with write access.
Merging is blocked

Merge pull request ▾ **A**

**Changes approved**
1 approving review by @mario
No conflicts with base branch

Merge pull request ▾ **B**

Sin aprobación: bloqueado.

Con aprobación: se puede unir.

5.2 · El conflicto (os va a pasar a casi todos)

Todos habéis añadido una línea **en el mismo sitio**: al final de la tabla. En cuanto entra la primera PR, en las demás aparece *This branch has conflicts that must be resolved*. **No es un error**. Git no puede adivinar qué fila va primero y te pregunta.

- 1 En tu PR, pulsa **Resolve conflicts** **C**.
- 2 Verás el fichero con unas marcas **D**. Entre <<<<<< y ===== está tu versión; entre ===== y >>>>>>, la que ya está en `main`.
- 3 **Decide:** aquí queremos las dos filas. Deja ambas y **borra las tres líneas de marcas**. Que no quede ni un <, = o > suelto.
- 4 **Mark as resolved** **E** → **Commit merge**.
- 5 Si la aprobación de tu compañero ha desaparecido, pídele que vuelva a mirar. Después, merge como en 5.1.



This branch has conflicts that must be resolved

Conflicting files: EQUIPO.md

Resolve conflicts

C

EQUIPO.md 1 conflict

✓ Mark as resolved

E

| Gabriel Mateos (profesor) | @gabrirobleda | - | Hacer preguntas incómodas |

<<<<<< equipo-lucia

← borrar

D

| Lucía Pérez | @luciaperez | A · Plataforma | Explicar con dibujos |

=====

← borrar

| Mario Sanz | @mariosanz | B · Datos | Encontrar bugs |

>>>>>> main

← borrar

Resultado correcto: la fila del profe, la de Mario y la de Lucía, una debajo de otra, sin marcas.

El editor de conflictos de GitHub. En azul, tu versión; en verde, la que ya está en main.

Una frase para recordar: resolver un conflicto es **decidir**, no arreglar un error. Hoy la decisión es fácil (las dos filas). Con código, a veces hay que hablar con la otra persona antes de decidir.

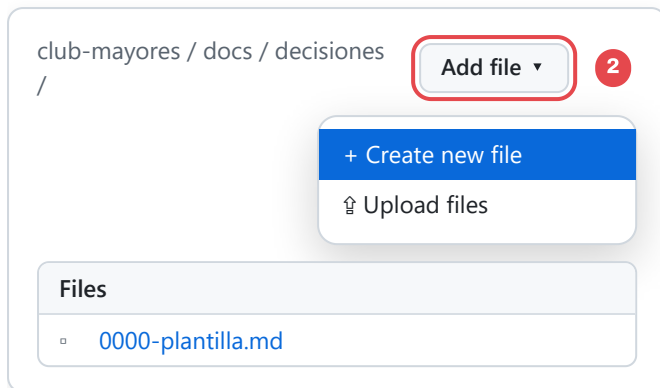
PUNTO DE CONTROL 5

- ☐ Mi PR está en estado **Merged** (morado) y la rama borrada.
- ☐ Si he tenido conflicto, lo he resuelto yo sin ayuda del profe.
- ☐ En `main`, `EQUIPO.md` tiene mi fila y no quedan marcas `<<<`.

6 El borrador de tu decisión

Objetivo: dejar preparado el documento que entregas la semana que viene, enlazado a tu issue, como pull request en borrador.

- 1 Abre `docs/decisiones/0000-plantilla.md`, pulsa el icono de copiar el contenido (📄 *Copy raw file*).
- 2 Vuelve a la carpeta `docs/decisiones/` → `Add file` → `Create new file` 2.
- 3 Nombre del fichero: el de tu tema en la tabla de la misión 2, por ejemplo `0001-plataforma.md` 3. Cada tema tiene su número para que no choquéis.
- 4 Pega la plantilla y rellena **solo** la cabecera y el apartado **Problema**. Lo demás es la tarea de esta semana.
- 5 `Commit changes...` → nueva rama `adr-0001-plataforma` → `Propose changes`.
- 6 En la descripción de la PR escribe `Closes #4` (el número de **tu** issue) 6. Esta vez sí: cuando se acepte la decisión, tu investigación estará terminada.
- 7 Despliega la flecha del botón verde y elige `Create draft pull request` 7. Un *draft* (borrador) dice "estoy trabajando en esto, aún no lo reviséis".
- 8 La semana que viene, cuando esté completo, pulsarás `Ready for review`.



Crear un fichero nuevo en la carpeta.



El nombre con la ruta, y la cabecera rellena.



Pull request en borrador, enlazada a tu issue.

¿Por qué así? Las decisiones técnicas (qué plataforma, dónde van los datos) cuestan mucho de cambiar después. Escribirlas, con alternativas y fuentes, y revisarlas como el código, evita que dentro de dos meses nadie recuerde por qué se hizo así. Y enlazar la PR con la issue hace que el tablero se actualice solo.

PUNTO DE CONTROL 6 · FINAL

- ☐ Existe una PR en borrador con mi fichero `docs/decisiones/000X-tema.md`.
- ☐ Su descripción dice `Closes #` y el número de mi issue.
- ☐ En la PR, a la derecha, en **Development**, aparece mi issue enlazada.

Criterio de éxito del equipo

Antes del próximo miércoles: **EQUIPO.md con todas las filas en main** y sin marcas de conflicto · **una PR fusionada por persona**, aprobada por otra · **una revisión con comentario por persona** · **una issue de investigación por persona** en el tablero · **una PR en borrador por persona** con su decisión. Cuando esté todo, cerrad la issue #1 (a mano: botón `Close issue`).

+ Retos extra

si acabas antes

Para quien termine antes. En cualquier orden.

- 1 Mejora la guía.** Algo de hoy que no estaba en `GUIA_GITHUB.md` y te habría ayudado: añádelo con una PR (rama, revisión, merge: el ciclo completo otra vez).

- 2 **Pulsa la tecla punto** (.) dentro del repositorio. Se abre **github.dev**, un VS Code ligero en el navegador. ¿Qué diferencia hay con un Codespace? (Pista: ¿tiene terminal?) Apúntalo en el cuaderno.
- 3 **La historia del repositorio.** Pestaña **Insights** → **Network**. Verás las ramas de hoy y cómo se unieron a main. Y en **Contributors**, quién ha hecho qué. (Así evalúa el profe.)
- 4 **Blame.** Abre **EQUIPO0.md** y pulsa **Blame**. ¿Quién escribió cada línea y en qué commit? Busca qué significa "blame" y por qué es útil (y por qué se llama así).
- 5 **Tu perfil.** Crea un repositorio con el mismo nombre que tu usuario (**tuusuario/tuusuario**) con un **README.md**: aparecerá en tu perfil de GitHub. Es tu escaparate para prácticas y empresas.
- 6 **GitHub Education.** Entra en **education.github.com** y comprueba qué ofrece a estudiantes (Copilot, más horas de Codespaces...). Si te interesa, solicítalo con tu correo del centro. Cuéntalo al equipo la semana que viene.

Soluciones de la búsqueda del tesoro (misión 1)

1. Un chat. No competimos con WhatsApp: le quitamos lo que hace mal.
2. Cuando el admin publica una actividad, el móvil de una persona mayor recibe el aviso, responde y el admin lo ve. Sí o no, sin nota parcial.
3. No. Borrarla en un commit nuevo no basta: sigue en el historial. Hay que avisar (y cambiar la clave).
4. **git status**.
5. **Closes #N**, con el número de la issue.
6. El profesor, que hace de cliente. Sabe hacer preguntas incómodas.
7. "Me lo ha dicho una IA". Si la usas, compruébalo en la fuente y enlaza la fuente.
8. Nada de nombres: iniciales y edad.

Chuleta para el resto del curso

QUIERO...	HAGO...
Empezar una tarea	Issue asignada → tarjeta a "En curso" → rama issue-N-algo
Proponer un cambio	Commit en mi rama → PR con Closes #N → revisor
Avisar de que aún no está	PR en borrador (<i>draft</i>)
Ver qué me toca revisar	Pull requests → review-requested:@me
Arreglar lo que me piden en una revisión	Nuevo commit en la misma rama: la PR se actualiza sola
Resolver un conflicto	Resolve conflicts → decidir → borrar marcas → Mark as resolved → Commit merge
Registrar una decisión	docs/decisiones/000N-tema.md desde la plantilla, por PR